FATEC JACAREÍ

Prof. Francisco de Moura

RAFAEL VITOR

MATHEUS VENANCIO

Cadastro de pessoas com TDD

CRUD em JavaScript com cálculo de idade, testes automatizados, análise estática e imagem Docker

JACAREÍ

2026

SUMÁRIO

1 INTRODUÇÃO 3

2 VISÃO GERAL DA SOLUÇÃO 4

2.1 Tecnologias utilizadas 4

2.2 Organização em camadas 4

2.3 Estrutura de pastas 5

3 A APLICAÇÃO EM FUNCIONAMENTO 6

3.1 Cadastro 6

3.2 Validações 8

3.3 Edição e exclusão 10

3.4 Uso no celular 12

4 REGRAS DE NEGÓCIO 14

4.1 Validação do CPF 14

4.2 Cálculo da idade 14

4.3 Regras de validação 15

5 DESENVOLVIMENTO ORIENTADO A TESTES 16

5.1 O ciclo do TDD 16

5.2 Como aplicamos no projeto 16

5.3 Exemplo de um ciclo 16

6 TESTES AUTOMATIZADOS 18

6.1 Testes unitários 18

6.2 Testes de integração 20

6.3 Cobertura 21

7 API REST 23

8 QUALIDADE DO CÓDIGO 25

8.1 Análise estática com ESLint 25

8.2 Formatação com Prettier 25

9 VERSIONAMENTO E PULL REQUEST 26

9.1 Fluxo de trabalho 26

9.2 Pull request 26

9.3 Integração contínua 26

10 IMAGEM DOCKER 28

11 CONSIDERAÇÕES FINAIS 29

REFERÊNCIAS 30

1 INTRODUÇÃO

Este trabalho descreve o desenvolvimento de uma aplicação de cadastro de pessoas em JavaScript. O sistema permite cadastrar, consultar, alterar e excluir pessoas a partir de três dados (CPF, nome e data de nascimento) e calcula a idade de cada uma com base na data de nascimento.

Além do CRUD, a atividade pedia que o desenvolvimento seguisse algumas práticas de engenharia de software. Os itens solicitados foram:

- desenvolvimento orientado a testes (TDD);
- controle de versão;
- abertura de *pull request*;
- análise estática do código;
- formatação padronizada;
- testes unitários;
- testes de integração;
- criação de uma imagem da aplicação.

A seção 2 apresenta as tecnologias e a organização do projeto. A seção 3 mostra a aplicação funcionando. As seções 4 a 6 tratam das regras de negócio, de como aplicamos o TDD e dos testes automatizados. As seções 7 a 10 cobrem a API, a qualidade do código, o versionamento e a imagem Docker, e a seção 11 traz nossas considerações finais.

2 VISÃO GERAL DA SOLUÇÃO

2.1 Tecnologias utilizadas

Escolhemos o Node.js 22 porque ele já traz, sem instalar nada, um servidor HTTP (`node:http`) e um executor de testes (`node:test`). Com isso a aplicação não tem nenhuma dependência de produção. As únicas bibliotecas instaladas são de desenvolvimento e servem para a análise estática e a formatação. O Quadro 1 resume o que usamos em cada parte.

Quadro 1 – Ferramentas utilizadas

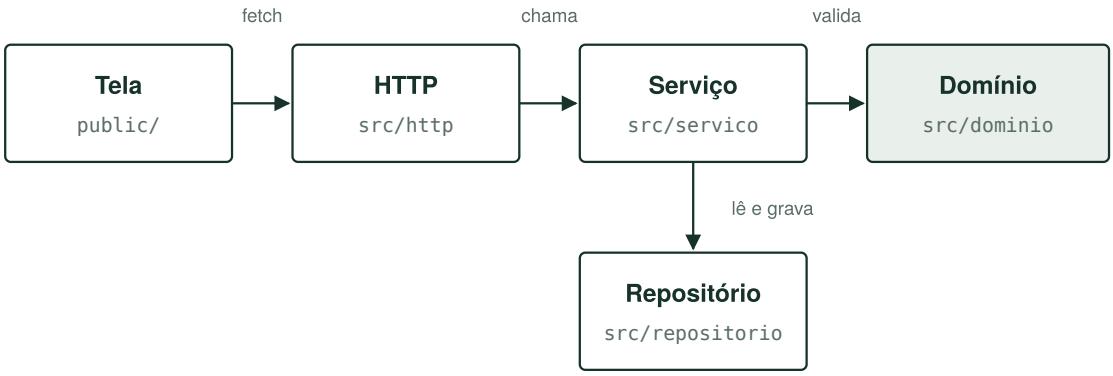
Ferramenta	Para que usamos
Node.js 22	Execução da aplicação e do servidor HTTP
`node:test` e `node:assert`	Testes unitários e de integração
HTML, CSS e JavaScript	Tela de cadastro, sem framework
ESLint 9	Análise estática
Prettier 3	Formatação do código
Git e GitHub	Controle de versão e *pull request*
GitHub Actions	Integração contínua
Docker	Imagem da aplicação

Fonte: elaborado pelos autores (2026).

2.2 Organização em camadas

Dividimos o código em quatro camadas, cada uma numa pasta dentro de `src`. A intenção foi deixar as regras de negócio separadas dos detalhes de HTTP e de armazenamento, porque assim conseguimos testar essas regras sem subir servidor nenhum. A Figura 1 mostra como as camadas se comunicam.

Figura 1 – Camadas da aplicação



Fonte: elaborado pelos autores (2026).

A camada de **domínio** concentra a validação do CPF, o cálculo da idade e a validação dos dados de uma pessoa. São funções puras: recebem valores e devolvem resultados, sem acessar rede, disco ou o relógio do sistema. O **repositório** guarda as pessoas, nesta versão em memória. O **serviço** aplica as regras do cadastro, como impedir CPF repetido, e acrescenta a idade a cada leitura. A camada **HTTP** transforma requisições em chamadas ao serviço e erros em códigos de status. A **tela** é uma página estática que conversa com a API usando `fetch`.

Um detalhe que fez diferença nos testes foi o relógio. O serviço recebe uma função `relogio` que devolve a data atual. Na aplicação ela é apenas `() => new Date()`, mas nos testes passamos uma data fixa. Dessa forma, um teste que hoje espera "31 anos" continua passando no ano que vem.

2.3 Estrutura de pastas

```
cadastro-pessoas/
├── src/
│   ├── dominio/
│   │   ├── cpf.js                  validação, normalização e máscara do CPF
│   │   ├── idade.js                leitura de datas e cálculo da idade
│   │   ├── pessoa.js               validação dos dados de uma pessoa
│   │   └── erros.js                erros de validação, não encontrado e conflito
│   ├── repositorio/repositorioEmMemoria.js
│   ├── servico/pessoaServico.js
│   ├── http/app.js                 rotas, leitura do JSON e códigos de status
│   └── server.js                   ponto de entrada
├── public/                         index.html, styles.css e app.js da tela
├── test/
│   ├── unit/                       um arquivo de teste por módulo
│   └── integration/api.test.js
├── .github/                        pipeline do CI e modelo de pull request
├── Dockerfile
├── eslint.config.js
├── .prettierrc.json
└── package.json
```

3 A APLICAÇÃO EM FUNCIONAMENTO

As capturas desta seção foram feitas com a aplicação rodando localmente (`npm start`) e aberta no navegador em `http://localhost:3000`, no dia 16 de outubro de 2026. A data importa porque as idades mostradas são calculadas em relação a ela.

3.1 Cadastro

Ao abrir a aplicação sem nenhum cadastro, a lista mostra uma orientação para começar pela ficha (Figura 2). Enquanto o usuário digita, o campo de CPF aplica a máscara automaticamente, e a data é escolhida pelo seletor do próprio navegador (Figura 3).

Figura 2 – Tela inicial, antes de qualquer cadastro



Fonte: elaborado pelos autores (2026).

Figura 3 – Ficha preenchida, com a máscara do CPF aplicada



Fonte: elaborado pelos autores (2026).

Depois de salvar, a pessoa aparece na lista em ordem alfabética, com a idade em destaque à direita (Figura 4). Escolhemos as datas de propósito para mostrar os casos de borda: João Pedro faz 18 anos no próprio dia da captura; Ana Beatriz, que faz aniversário no dia seguinte, ainda aparece com 25; e Luiza, nascida em 29/02/2024, aparece com 2 anos.

Figura 4 – Lista com cinco pessoas cadastradas

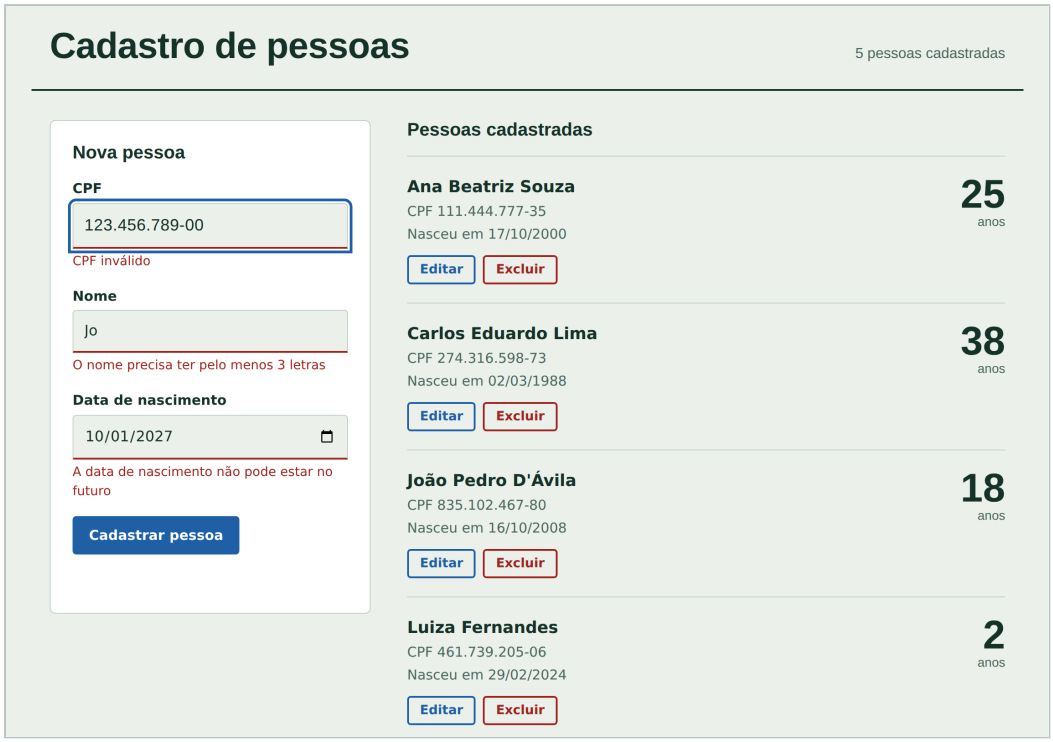


Fonte: elaborado pelos autores (2026).

3.2 Validações

Quando algum campo está errado, a API devolve uma mensagem para cada campo e a tela mostra todas de uma vez, logo abaixo do campo correspondente. O cursor vai para o primeiro campo com problema (Figura 5).

Figura 5 – Erros de CPF, nome e data de nascimento exibidos juntos



Fonte: elaborado pelos autores (2026).

Tentar cadastrar um CPF que já existe gera a resposta 409 (conflito), exibida no próprio campo de CPF (Figura 6). A comparação é feita só com os dígitos, então digitar o mesmo CPF com ou sem pontos não engana a verificação.

Figura 6 – Tentativa de cadastrar um CPF repetido



Fonte: elaborado pelos autores (2026).

3.3 Edição e exclusão

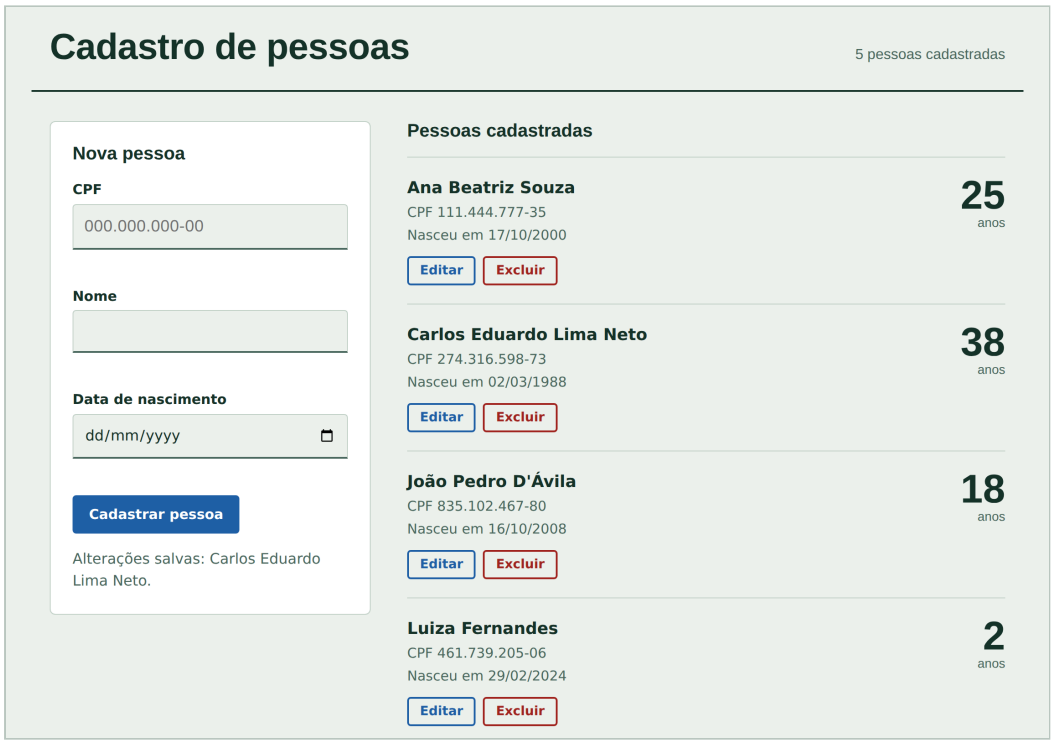
O botão Editar leva os dados da pessoa para a ficha, que muda de título e passa a ter os botões “Salvar alterações” e “Cancelar edição”. O CPF fica bloqueado, porque é ele que identifica a pessoa (Figura 7). Ao salvar, a lista é atualizada e uma mensagem confirma a alteração (Figura 8).

Figura 7 – Edição de um cadastro, com o CPF bloqueado



Fonte: elaborado pelos autores (2026).

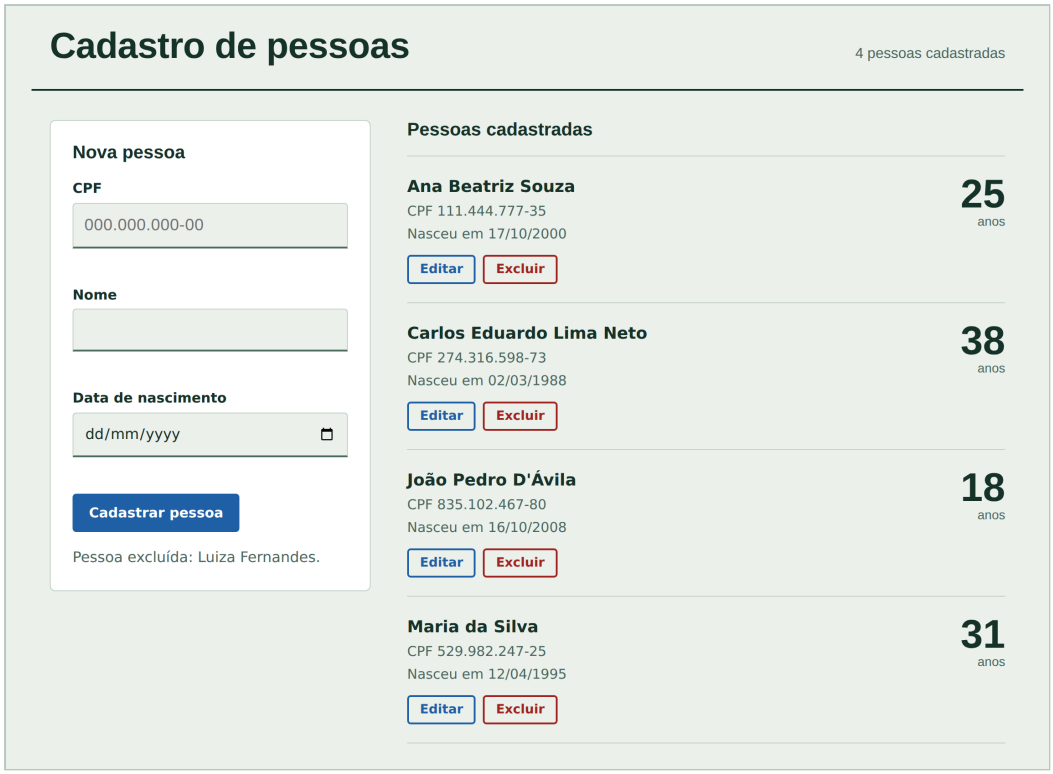
Figura 8 – Alteração salva



Fonte: elaborado pelos autores (2026).

A exclusão pede confirmação antes de apagar. Na Figura 9, Luiza Fernandes já foi removida e o contador do topo passou para quatro pessoas.

Figura 9 – Lista depois de excluir um cadastro

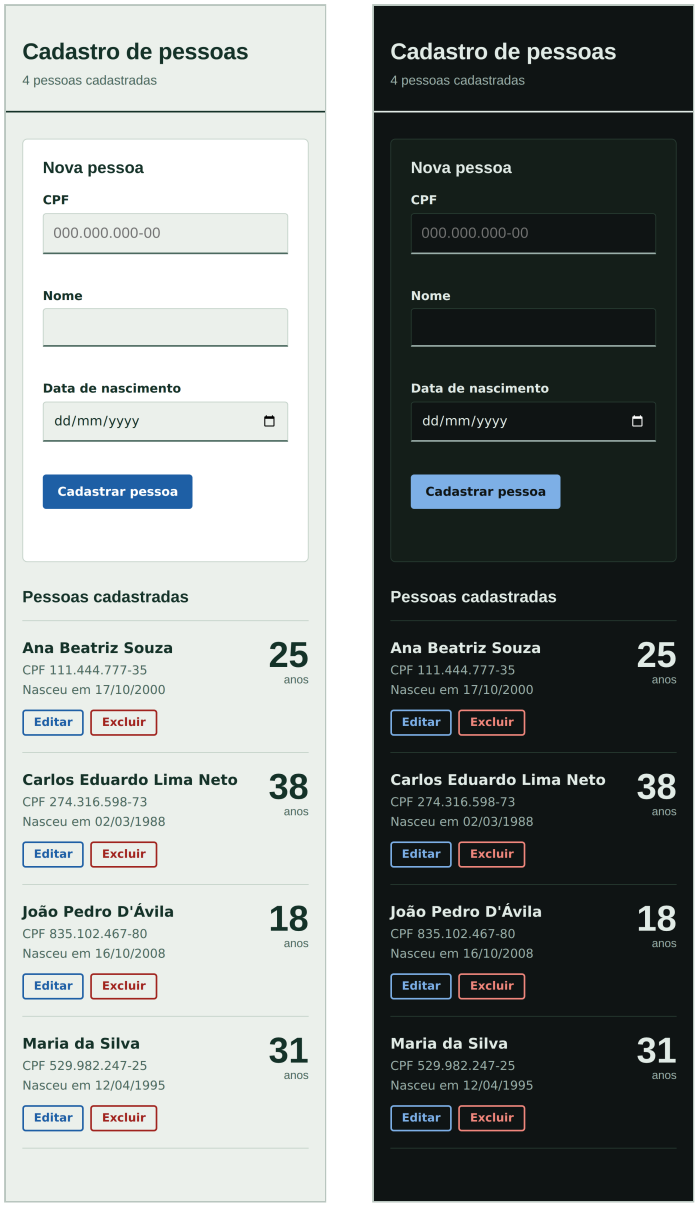


Fonte: elaborado pelos autores (2026).

3.4 Uso no celular

Em telas estreitas a ficha e a lista ficam em uma coluna só, e as cores acompanham o tema claro ou escuro escolhido no aparelho (Figura 10).

Figura 10 – A mesma tela no celular, nos temas claro e escuro



(a) tema claro (b) tema escuro

Fonte: elaborado pelos autores (2026).

4 REGRAS DE NEGÓCIO

4.1 Validação do CPF

O CPF tem 11 dígitos. Os nove primeiros formam o número base e os dois últimos são dígitos verificadores, calculados a partir dos anteriores. Para o primeiro verificador, cada um dos nove dígitos é multiplicado por um peso que vai de 10 a 2, e os produtos são somados. O Quadro 2 mostra a conta para o CPF 529.982.247-25.

Quadro 2 – Cálculo do primeiro dígito verificador de 529.982.247-25

Dígito	**5**	**2**	**9**	**9**	**8**	**2**	**2**	**4**	**7**	**Soma**
Peso	10	9	8	7	6	5	4	3	2	
Produto	50	18	72	63	48	10	8	12	14	**295**

Fonte: elaborado pelos autores (2026).

Multiplicamos a soma por 10 e pegamos o resto da divisão por 11: 2950 dividido por 11 deixa resto 2, que é justamente o primeiro verificador. Se o resto der 10, o dígito vira 0. O segundo verificador segue a mesma conta, agora com os dez primeiros dígitos (incluindo o 2 que acabou de ser calculado) e pesos de 11 a 2. A soma dá 347, e 3470 dividido por 11 deixa resto 5. No código, as duas contas usam a mesma função:

```
function calcularDigitoVerificador(digitos, pesoInicial) {
  const soma = digitos.reduce(
    (total, digito, indice) => total + digito * (pesoInicial - indice),
    0,
  );
  const resto = (soma * 10) % 11;
  return resto === 10 ? 0 : resto;
}
```

Além da conta, recusamos duas situações que passariam despercebidas. A primeira são os CPFs com todos os dígitos iguais, como 111.111.111-11, que passam no cálculo mas não são válidos. A segunda são valores enviados como número em vez de texto: um número perde os zeros à esquerda, e 000.000.001-91, por exemplo, é um CPF válido.

4.2 Cálculo da idade

A idade é a diferença entre o ano atual e o ano de nascimento, menos um se o aniversário ainda não chegou naquele ano. A comparação é feita por mês e dia, sem converter as datas para milissegundos. Isso evita erros de fuso horário e torna o comportamento com anos bissextos previsível: quem nasceu em 29 de fevereiro completa ano em 1º de março quando o ano não é bissexto. O Quadro 3 traz os casos que cobrimos nos testes.

Quadro 3 – Casos de cálculo da idade

Nascimento	Data de referência	Idade	Situação
10/03/2000	16/10/2026	26	Aniversário já passou no ano
16/10/2000	16/10/2026	26	Dia do aniversário
17/10/2000	16/10/2026	25	Véspera do aniversário
01/12/2000	16/10/2026	25	Aniversário em um mês mais à frente
16/10/2026	16/10/2026	0	Nasceu no dia
29/02/2000	28/02/2025	24	Ano não bissexto, antes de 1º de março
29/02/2000	01/03/2025	25	Ano não bissexto, a partir de 1º de março
29/02/2000	29/02/2024	24	Ano bissexto

Fonte: elaborado pelos autores (2026).

A idade não é gravada junto com o cadastro. Ela é calculada toda vez que a pessoa é lida, então nunca fica desatualizada: no dia seguinte ao aniversário, a lista já mostra a idade nova sem que ninguém precise editar nada.

4.3 Regras de validação

Os três campos são validados juntos, e a resposta traz o erro de cada campo de uma vez, em vez de parar no primeiro. O Quadro 4 lista as regras.

Quadro 4 – Regras de validação dos campos

Campo	Regras
CPF	Obrigatório; 11 dígitos, com ou sem máscara; dígitos verificadores corretos; não pode ter todos os dígitos iguais; único no cadastro; não pode ser alterado depois
Nome	Obrigatório; de 3 a 100 caracteres; apenas letras (com acento), espaços, hífen e apóstrofo; espaços repetidos são removidos
Data de nascimento	Obrigatória; formato AAAA-MM-DD; precisa existir no calendário (30/02 é recusado); a partir de 1900; não pode estar no futuro

Fonte: elaborado pelos autores (2026).

5 DESENVOLVIMENTO ORIENTADO A TESTES

5.1 O ciclo do TDD

No TDD, o código de produção só é escrito para fazer passar um teste que falhou antes. Beck (2002) descreve o processo como um ciclo de três etapas, que repetimos para cada comportamento novo:

1. **Red**: escrever um teste para o próximo comportamento e confirmar que ele falha;
2. **Green**: escrever o código mínimo para o teste passar;
3. **Refactor**: melhorar o código mantendo todos os testes passando.

Ver o teste falhar primeiro não é só formalidade. Se ele já passa antes de existir a implementação, o teste não está verificando o que achamos que verifica.

5.2 Como aplicamos no projeto

Desenvolvemos de dentro para fora: primeiro as funções do domínio, que não dependem de nada, depois o serviço e, por último, a API. Em cada ciclo o teste foi para um commit próprio, antes do commit da implementação, para que o histórico do Git mostre a ordem em que as coisas aconteceram. O Quadro 5 mostra a sequência.

Quadro 5 – Sequência dos ciclos de TDD

#	Teste (red)	Implementação (green)
1	`cpf.test.js`	`dominio/cpf.js`
2	`idade.test.js`	`dominio/idade.js`
3	`pessoa.test.js`	`dominio/erros.js` e `dominio/pessoa.js`
4	`pessoaServico.test.js`	`repositorioEmMemoria.js` e `pessoaServico.js`
5	`api.test.js`	`http/app.js`, `server.js` e a pasta `public`

Fonte: elaborado pelos autores (2026).

5.3 Exemplo de um ciclo

Um bom exemplo é o caso de quem nasce em 29 de fevereiro. Primeiro escrevemos os testes que descrevem o comportamento esperado:

```
it('quem nasceu em 29/02 ainda não completou ano em 28/02 de ano não bissexto', () => {
  assert.equal(calcularIdade('2000-02-29', em(2025, 2, 28)), 24);
});

it('quem nasceu em 29/02 completa ano em 01/03 de ano não bissexto', () => {
  assert.equal(calcularIdade('2000-02-29', em(2025, 3, 1)), 25);
});
```

Sem a função `calcularIdade`, a execução falha. Em seguida escrevemos a implementação, comparando mês e dia em vez de subtrair datas:

```
export function calcularIdade(dataNascimento, referencia = new Date()) {
  const nascimento = interpretarData(dataNascimento);
  if (!nascimento) throw new TypeError(`Data de nascimento inválida: ${dataNascimento}`);

  const hoje = diaDoCalendario(referencia);
  if (compararDatas(nascimento, hoje) > 0) {
    throw new RangeError('A data de nascimento não pode estar no futuro');
  }

  const aniversarioAindaNaoChegou =
    hoje.mes < nascimento.mes || (hoje.mes === nascimento.mes && hoje.dia < nascimento.dia);

  return hoje.ano - nascimento.ano - (aniversarioAindaNaoChegou ? 1 : 0);
}
```

A data de referência entra como parâmetro, o que deixa o teste independente do dia em que é executado. O mesmo vale para o serviço, que recebe o relógio por injeção. O teste abaixo avança o relógio um dia e confere que a idade muda sozinha:

```
it('a idade acompanha o calendário sem precisar editar o cadastro', async () => {
  await servico.cadastrar(ANA); // faz aniversário em 17/10
  assert.equal((await servico.buscar(ANA.cpf)).idade, 25);

  agora = new Date(2026, 9, 17);
  assert.equal((await servico.buscar(ANA.cpf)).idade, 26);
});
```

6 TESTES AUTOMATIZADOS

Ao todo são 85 testes, 70 unitários e 15 de integração, escritos com o executor nativo do Node. Os comandos ficam no `package.json`: `npm run test:unit`, `npm run test:integration`, `npm test` para os dois juntos e `npm run test:coverage` para o relatório de cobertura.

6.1 Testes unitários

Os testes unitários verificam cada módulo isolado. O serviço é testado com o repositório em memória e um relógio fixo, sem servidor HTTP. O Quadro 6 mostra a distribuição, e as Figuras 11 e 12 mostram a execução completa.

Quadro 6 – Testes unitários por arquivo

Arquivo	Testes	O que verifica
`cpf.test.js`	17	Dígitos verificadores, resto 10, zeros à esquerda, dígitos repetidos, máscara
`idade.test.js`	18	Aniversário, véspera, 29 de fevereiro, datas inexistentes e no futuro
`pessoa.test.js`	20	Regras de cada campo e retorno dos erros todos juntos
`pessoaServico.test.js`	15	CPF único, ordem alfabética, edição sem trocar CPF, exclusão, idade na leitura

Fonte: elaborado pelos autores (2026).

Figura 11 – Execução dos testes unitários (parte 1: CPF e idade)



Fonte: elaborado pelos autores (2026).

Figura 12 – Execução dos testes unitários (parte 2: pessoa e serviço)



Fonte: elaborado pelos autores (2026).

6.2 Testes de integração

Os testes de integração sobem o servidor HTTP de verdade numa porta livre (porta 0, que o sistema escolhe) e fazem requisições com `fetch`, conferindo status, cabeçalhos e corpo das respostas. Cada teste começa com um servidor novo e um repositório vazio, então um teste

não interfere no outro. Além do CRUD completo, cobrimos JSON malformado, corpo grande demais (413), método não suportado (405) e tentativas de acessar arquivos fora da pasta `public` (Figura 13).

Figura 13 – Execução dos testes de integração

```
Terminal — testes de integração                                    bash

~/cadastro-pessoas$ npm run test:integration
> cadastro-pessoas@1.0.0 test:integration
> node --test "test/integration/**/*.test.js"

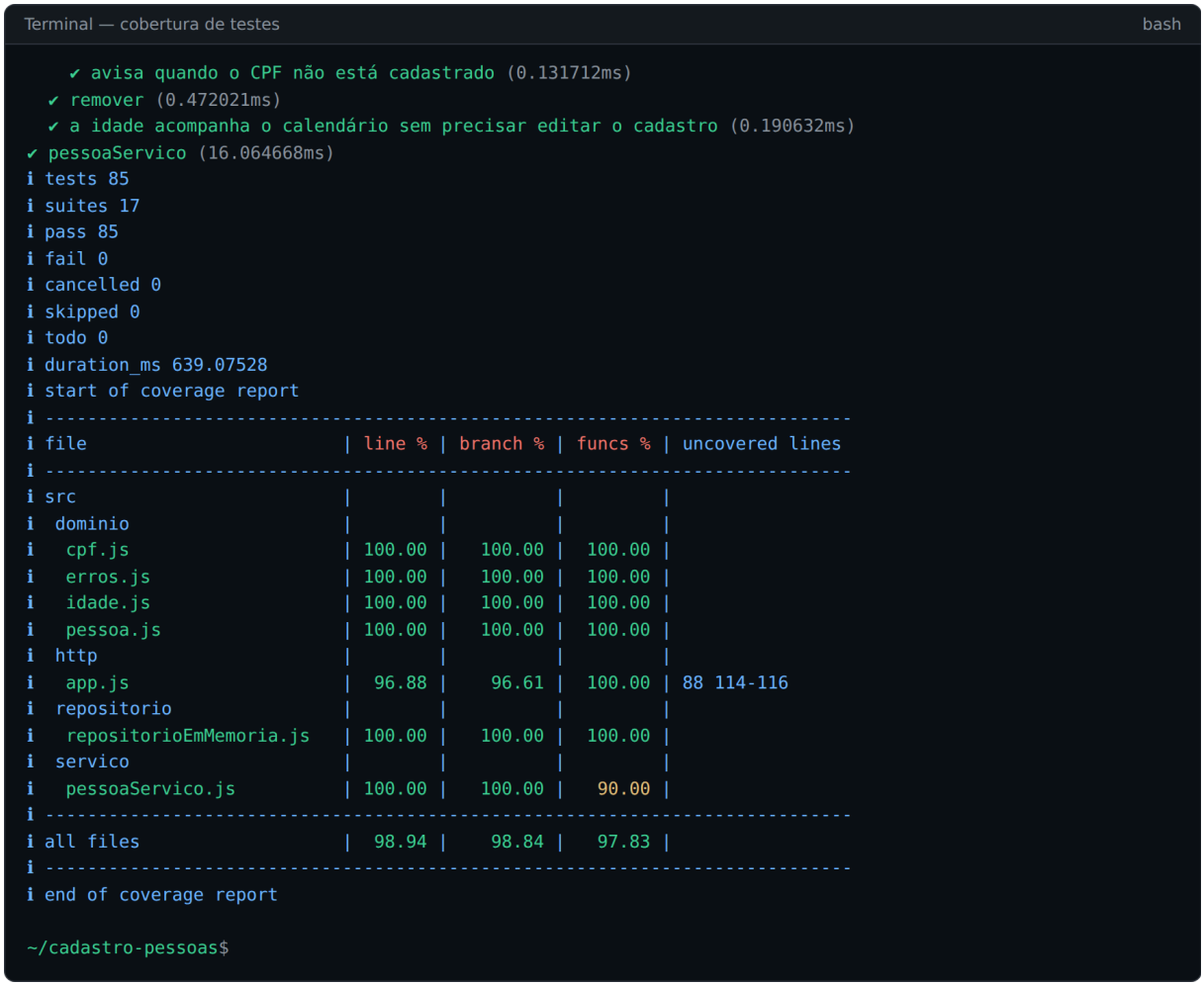
▶ API /api/pessoas (servidor HTTP real)
  ✔ GET /health responde ok (30.353959ms)
  ✔ faz o CRUD completo: cadastrar, consultar, listar, atualizar e remover (32.977397ms)
  ✔ aceita o CPF com máscara no endereço (18.672968ms)
  ✔ POST com dados inválidos responde 400 com o erro de cada campo (13.615527ms)
  ✔ POST com CPF já cadastrado responde 409 (13.999758ms)
  ✔ POST com JSON quebrado responde 400 (11.118842ms)
  ✔ POST sem corpo ou com algo que não é objeto responde 400 (11.761804ms)
  ✔ POST com corpo grande demais responde 413 (12.992414ms)
  ✔ PUT tentando trocar o CPF responde 400 (12.116482ms)
  ✔ PUT e DELETE de CPF inexistente respondem 404 (4.856362ms)
  ✔ método não suportado responde 405 com os permitidos (14.778203ms)
  ✔ CPF mal codificado no endereço responde 400 (3.44893ms)
  ✔ rota desconhecida responde 404 (6.44929ms)
  ✔ não serve arquivos fora da pasta public (5.026811ms)
  ✔ serve a página do cadastro em / (4.429132ms)
✔ API /api/pessoas (servidor HTTP real) (198.568587ms)
ℹ tests 15
ℹ suites 1
ℹ pass 15
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 297.941979
~/cadastro-pessoas$
```

Fonte: elaborado pelos autores (2026).

6.3 Cobertura

O próprio Node gera o relatório de cobertura com a opção `--experimental-test-coverage` (Figura 14). O domínio, o repositório e o serviço ficaram com 100% das linhas cobertas. As poucas linhas sem cobertura em `app.js` são a resposta 405 da rota de uma pessoa específica e o tratamento de erro interno (500), que só acontece em falhas inesperadas.

Figura 14 – Relatório de cobertura dos testes



Fonte: elaborado pelos autores (2026).

7 API REST

A tela conversa com o servidor por uma API REST. O CPF pode ser enviado com ou sem máscara, inclusive no endereço. O Quadro 7 lista as rotas.

Quadro 7 – Rotas da API

Método	Rota	Resposta
`GET`	`/api/pessoas`	200 com a lista em ordem alfabética
`GET`	`/api/pessoas/:cpf`	200 com a pessoa, ou 404
`POST`	`/api/pessoas`	201 com a pessoa criada, 400 se os dados forem inválidos ou 409 se o CPF já existir
`PUT`	`/api/pessoas/:cpf`	200 com a pessoa atualizada, 400 ou 404
`DELETE`	`/api/pessoas/:cpf`	204 sem corpo, ou 404
`GET`	`/health`	200 com `{"status":"ok"}`, usado pelo Docker

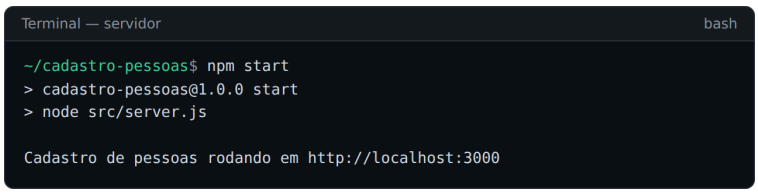
Fonte: elaborado pelos autores (2026).

Quando os dados são inválidos, a resposta traz o erro de cada campo no objeto `detalhes`, que é o que a tela usa para mostrar as mensagens embaixo dos campos:

```
{
  "erro": "Dados inválidos",
  "detalhes": {
    "cpf": "CPF inválido",
    "nome": "Informe o nome",
    "dataNascimento": "A data de nascimento não pode estar no futuro"
  }
}
```

Além dos testes de integração, testamos a API manualmente pelo terminal. A Figura 15 mostra o servidor no ar, e a Figura 16 mostra o ciclo completo feito com `curl`: cadastro, consulta, CPF repetido, dados inválidos, alteração, exclusão e a consulta final, que já retorna 404.

Figura 15 – Servidor iniciado com `npm start`



Fonte: elaborado pelos autores (2026).

Figura 16 – Chamadas à API feitas com `curl`

Terminal — chamadas à API com curl bash

```
~/cadastro-pessoas$ curl -s -X POST http://localhost:3000/api/pessoas -H "Content-Type: application/json" -d '{"c
pf":"529.982.247-25","nome":"Maria da Silva","dataNascimento":"1995-04-12"}' -w "\nstatus %{http_code}\n"
{"cpf":"52998224725","nome":"Maria da Silva","dataNascimento":"1995-04-12","idade":31}
status 201
~/cadastro-pessoas$ curl -s http://localhost:3000/api/pessoas/529.982.247-25 -w "\nstatus %{http_code}\n"
{"cpf":"52998224725","nome":"Maria da Silva","dataNascimento":"1995-04-12","idade":31}
status 200
~/cadastro-pessoas$ curl -s -X POST http://localhost:3000/api/pessoas -H "Content-Type: application/json" -d '{"c
pf":"529.982.247-25","nome":"Maria da Silva","dataNascimento":"1995-04-12"}' -w "\nstatus %{http_code}\n"
{"erro":"Já existe uma pessoa com este CPF"}
status 409
~/cadastro-pessoas$ curl -s -X POST http://localhost:3000/api/pessoas -H "Content-Type: application/json" -d '{"c
pf":"123","nome":"","dataNascimento":"2030-01-01"}' -w "\nstatus %{http_code}\n"
{"erro":"Dados inválidos","detalhes":{"cpf":"CPF inválido","nome":"Informe o nome","dataNascimento":"A data de na
scimento não pode estar no futuro"}}
status 400
~/cadastro-pessoas$ curl -s -X PUT http://localhost:3000/api/pessoas/52998224725 -H "Content-Type: application/js
on" -d '{"nome":"Maria Souza","dataNascimento":"1990-01-01"}' -w "\nstatus %{http_code}\n"
{"cpf":"52998224725","nome":"Maria Souza","dataNascimento":"1990-01-01","idade":36}
status 200
~/cadastro-pessoas$ curl -s -X DELETE http://localhost:3000/api/pessoas/52998224725 -w "status %{http_code}\n"
status 204
~/cadastro-pessoas$ curl -s http://localhost:3000/api/pessoas/52998224725 -w "\nstatus %{http_code}\n"
{"erro":"Pessoa não encontrada"}
status 404
~/cadastro-pessoas$
```

Fonte: elaborado pelos autores (2026).

8 QUALIDADE DO CÓDIGO

8.1 Análise estática com ESLint

Partimos da configuração recomendada do ESLint e acrescentamos algumas regras que consideramos importantes para o projeto (Quadro 8). A pasta `public` recebe as variáveis globais do navegador, como `document`, e o restante do projeto recebe as do Node. A análise roda com `npm run lint`, e `npm run lint:fix` corrige automaticamente o que for possível.

Quadro 8 – Regras do ESLint adicionadas à configuração recomendada

Regra	Motivo
`eqeqeq`	Obriga `===` e `!==`, evitando conversões automáticas de tipo
`no-var`, `prefer-const`	Só `let` e `const`, e `const` sempre que o valor não muda
`no-unused-vars`	Aponta variáveis e imports esquecidos
`no-shadow`	Impede declarar uma variável com o mesmo nome de outra de um escopo de fora
`no-param-reassign`	Impede alterar os parâmetros recebidos por uma função
`complexity` (10), `max-depth` (3)	Avisam quando uma função fica com caminhos ou aninhamentos demais

Fonte: elaborado pelos autores (2026).

8.2 Formatação com Prettier

A formatação fica a cargo do Prettier, para que nenhum de nós precise discutir aspas ou ponto e vírgula em revisão de código. A configuração é curta:

```
{
  "singleQuote": true,
  "semi": true,
  "trailingComma": "all",
  "printWidth": 100,
  "endOfLine": "lf"
}
```

O `.editorconfig` repete as regras básicas (UTF-8, dois espaços, quebra de linha LF) para o editor já salvar no padrão. O `.gitattributes` força o LF mesmo no Windows, para o Prettier não acusar diferença só por causa do sistema operacional. O comando `npm run format` formata tudo, e `npm run format:check` apenas confere, que é o que o CI executa. Para rodar formatação, análise estática e testes em sequência, usamos `npm run check`.

9 VERSIONAMENTO E PULL REQUEST

9.1 Fluxo de trabalho

O projeto é versionado com Git e hospedado no GitHub. Trabalhamos com duas branches: a `main`, que guarda a versão estável, e a `feature/crud-pessoas`, onde o CRUD foi desenvolvido. Na `main` entrou diretamente só o commit inicial de configuração; todo o código da funcionalidade chega até ela pelo *pull request*.

As mensagens de commit seguem o padrão Conventional Commits, escritas em português (Quadro 9). O prefixo deixa claro, só de olhar o histórico, o que é teste, o que é funcionalidade e o que é configuração.

Quadro 9 – Tipos de commit utilizados

Tipo	Uso	Exemplo
`chore`	Configuração do projeto	chore: configura projeto com ESLint, Prettier e editorconfig
`test`	Teste novo (etapa red)	test: validação de CPF pelos dígitos verificadores
`feat`	Implementação (etapa green)	feat: valida, normaliza e formata CPF
`build`	Imagem Docker	build: imagem Docker da aplicação
`ci`	Pipeline do GitHub Actions	ci: formatação, análise estática, testes e imagem Docker

Fonte: elaborado pelos autores (2026).

9.2 Pull request

O *pull request* leva a `feature/crud-pessoas` para a `main`. Criamos um modelo em `.github/pull_request_template.md`, que o GitHub preenche automaticamente ao abrir o PR. Ele tem um resumo do que muda, os comandos para testar localmente e um checklist com os itens da atividade: testes escritos antes da implementação, formatação, análise estática, testes unitários e de integração e a imagem respondendo em `/health`.

9.3 Integração contínua

A cada *pull request*, e a cada envio para a `main`, o GitHub Actions executa o pipeline descrito em `.github/workflows/ci.yml`, dividido em dois jobs (Quadro 10). O segundo só roda se o primeiro passar, e qualquer etapa com falha deixa o PR marcado como reprovado.

Quadro 10 – Jobs do pipeline de integração contínua

Job	Etapas
Formatação, análise estática e testes	Instala as dependências com `npm ci`, confere a formatação, roda o ESLint, os testes unitários, os de integração e o relatório de cobertura
Imagem Docker	Constrói a imagem, sobe um contêiner e espera `/health` responder. Na `main`, publica a imagem no GitHub Container Registry

Fonte: elaborado pelos autores (2026).

10 IMAGEM DOCKER

A imagem é gerada a partir do `Dockerfile` na raiz do projeto:

```
FROM node:22-alpine

WORKDIR /app

ENV NODE_ENV=production \
    PORT=3000 \
    TZ=America/Sao_Paulo

COPY package.json ./
COPY src ./src
COPY public ./public

USER node

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node -e "fetch('http://localhost:3000/health').then((r) => process.exit(r.ok ? 0 :
1)).catch(() => process.exit(1))"

CMD ["node", "src/server.js"]
```

Algumas escolhas merecem explicação. Usamos a variante `alpine` da imagem oficial do Node por ser bem menor. Não há `npm install`, porque a aplicação não tem dependências de produção. Copiamos só o que a aplicação precisa para rodar, e o `.dockerignore` deixa de fora testes, a pasta `.git` e a documentação. O contêiner roda com o usuário `node`, e não como `root`. O `HEALTHCHECK` consulta a rota `/health` para o Docker saber se a aplicação está respondendo. Por fim, a variável `TZ` coloca o contêiner no horário de Brasília, para a idade virar à meia-noite daqui e não à meia-noite de Greenwich.

Para construir e executar a imagem:

```
docker build -t cadastro-pessoas .
docker run --rm -p 3000:3000 cadastro-pessoas
```

Depois disso a aplicação fica disponível em `http://localhost:3000`, igual à execução com `npm start`.

11 CONSIDERAÇÕES FINAIS

A aplicação atende aos itens pedidos na atividade: o CRUD de pessoas com cálculo da idade, desenvolvido com TDD, versionado com Git, integrado por *pull request*, com análise estática, formatação padronizada, testes unitários e de integração e uma imagem Docker testada no CI.

O que mais aprendemos foi o valor de escrever o teste antes. Ter que descrever o resultado esperado obrigou a pensar logo no começo em situações que normalmente só apareceriam com o sistema em uso, como o aniversário de quem nasceu em 29 de fevereiro, o CPF que começa com zero e o cadastro feito no próprio dia do aniversário. Separar o domínio do HTTP também ajudou bastante: a maior parte dos testes roda em milissegundos, sem servidor.

O projeto tem limitações conhecidas. Os dados ficam em memória e se perdem quando o servidor reinicia, e não há autenticação nem paginação da lista. Como o serviço depende só dos métodos do repositório (`listar`, `buscar`, `salvar` e `remover`), trocar a memória por um banco como SQLite ou PostgreSQL exige apenas uma nova implementação desse repositório, sem mudar o restante. Outros próximos passos seriam busca por nome e testes de ponta a ponta na tela.

REFERÊNCIAS

BECK, Kent. **Test-driven development**: by example. Boston: Addison-Wesley, 2002.

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. Disponível em: https://www.conventionalcommits.org/pt-br/v1.0.0/. Acesso em: 16 out. 2026.

DOCKER. **Dockerfile reference**. Disponível em: https://docs.docker.com/reference/dockerfile/. Acesso em: 16 out. 2026.

ESLINT. **Configure ESLint**. Disponível em: https://eslint.org/docs/latest/use/configure/. Acesso em: 16 out. 2026.

GITHUB. **Documentação do GitHub Actions**. Disponível em: https://docs.github.com/pt/actions. Acesso em: 16 out. 2026.

NODE.JS. **Test runner**. Disponível em: https://nodejs.org/api/test.html. Acesso em: 16 out. 2026.

PRETTIER. **Options**. Disponível em: https://prettier.io/docs/options. Acesso em: 16 out. 2026.